Last-Mile Delivery Tracker

System Design Write-up

1. Rate Calculation Engine

The Rate Engine evaluates shipping fees through a deterministic, multi-variable pipeline. Inputs include package dimensions (Length, Breadth, Height in cm), actual weight (kg), pickup/drop pincodes, order classification (B2C or B2B), and payment mode (PREPAID or COD). Pickup and destination pincodes are resolved to operational zones via database lookups. The engine evaluates volumetric weight using the standard air cargo formula: Volumetric Weight = (Length × Breadth × Height) / 5000. Chargeable weight is selected as max(actual weight, volumetric weight). Zone relation is determined as INTRA if pickup and drop zones match, or INTER for cross-zone routes. The system queries the RateCard database for the matching order type and zone relation where min_weight ≤ chargeable weight < max_weight (or unbounded top slab). Base freight is calculated as base_price + ((chargeable weight - min_weight) × rate_per_kg). For COD shipments, a surcharge is calculated using CODConfig rules (either a fixed flat fee or a percentage of the base freight). The final total charge equals base freight plus COD surcharge. Database-driven rate cards allow administrators to modify weight slabs dynamically without code updates.

2. Zone Detection

Zone detection maps physical postal pincodes to logical operational territories. The system maintains a relational Zone model linked to ZoneArea records containing unique 6-digit pincodes and city names. During order preview or creation, the service queries ZoneArea by pincode to retrieve the associated zone_id. If a pincode is not registered, an explicit unmapped pincode operational error is returned. This database-driven mapping approach was selected over dynamic polygon geocoding for speed, reliability, and ease of management. It enables logistics managers to redefine operational boundaries or assign new postal codes instantaneously through administrative controls without requiring external GIS API dependencies.

3. Auto-Assignment Logic

The Auto-Assignment Engine pairs pending orders with the optimal delivery agent based on geographic distance and workload metrics. When triggered, the engine filters agents registered in the order's pickup zone who are both active (is_active = true) and on duty (is_available = true). If no active agent exists in the pickup zone, the system executes a citywide fallback query across all active and available agents. For candidate agents with recorded coordinates, the engine computes great-circle distance to the pickup location using the Haversine formula: $d = 2R \cdot \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos(\phi1)\cos(\phi2)\sin^2(\Delta\lambda/2))})$ where R = 6371 km. Candidates are ranked by: (1) shortest Haversine distance, (2) lowest active order workload count, and (3) earliest account creation timestamp. The top-ranked agent is assigned, existing assignments are deactivated, and an immutable assignment record is saved. If no eligible agent is available, the order remains UNASSIGNED until an agent checks in or an admin intervenes.

4. Failed Delivery Handling

When a delivery attempt fails (e.g., recipient unreachable), the agent records a failure reason and updates the status from OUT_FOR_DELIVERY to FAILED. An automated notification alerts the customer. The Customer Portal prompts the user to select a new delivery date. Submitting the request creates an immutable RescheduleAttempt audit record, updates the order's scheduled delivery date, resets current_status back to BOOKED, and re-triggers the Haversine auto-assignment pipeline to assign an available agent for the new date. Historical traceability is preserved because past failed statuses and reschedule requests remain stored in immutable audit logs.

5. Architecture & Data Integrity

The application follows a decoupled layered architecture: React frontend, Express REST API, and SQLite database managed by Prisma ORM. Data integrity is enforced through strict relational foreign keys and transaction boundaries. To guarantee audit compliance, OrderStatusHistory is strictly insert-only at the database service layer, preventing status modification or record deletion.

6. Security & Role-Based Access

Authentication uses JSON Web Tokens (JWT) signed with HMAC-SHA256, alongside bcrypt password hashing (10 salt rounds). Role-Based Access Control (RBAC) middleware enforces granular route authorizations across CUSTOMER, AGENT, and ADMIN roles. Authorization is validated server-side on every request.

7. Conclusion

The Last-Mile Delivery Tracker delivers a robust, reliable logistics platform. Through automated volumetric pricing, database zone mapping, Haversine nearest-agent assignment, and immutable audit logs, the system ensures operational accuracy, efficiency, and compliance.